

基于大模型编码智能体的自进化 eBPF 网络入侵防御系统

硕士学位论文选题报告

*A Self-Evolving eBPF Network Intrusion Prevention System
Driven by Large Language Model Coding Agents*

研究生	李兆曦	学科 (专业)	计算机科学与技术 (0812)
研究方向	网络空间安全 / 网络协议安全	指导教师	徐恪教授
论文类型	应用研究 / 应用基础研究	课题来源	国家级科研项目 / 自选
		选题报告日期	2026 年 6 月

摘要

随着 TCP/IP 协议栈跨层交互歧义持续催生链外 (off-path) 协议攻击, 而现有防御零散、滞后且高度依赖人工编写内核代码, 本课题提出并构建一个由大模型编码智能体自主维护、运行于内核的自进化 eBPF 网络入侵防御系统。受“超越梯度的学习 (Learning Beyond Gradients)”启发, 本文提出防御演化 (Defensive Evolution) 范式: 将防御能力沉淀为内核验证器 (verifier) 证明安全的 eBPF 代码, 而非神经网络权重; 由编码智能体以新颖性与覆盖度引导的主动探索探查攻击面、进化式合成防御程序, 并经“编译—验证—沙箱”可信闭环热加载到 XDP 数据面, 从而持续、可审计、形式化可验证地进化防御能力。本文将作者在 off-path 协议攻击上积累的攻击者视角反向用于防御, 构建攻防协同进化评测, 量化系统对已知与未知协议攻击的防御效果、误报率与性能开销。

关键词: 网络协议安全; eBPF; 大模型智能体; 防御演化; 自进化系统; 入侵防御

Abstract

Ambiguities in cross-layer TCP/IP interactions keep spawning off-path protocol attacks, yet existing defenses are fragmented, reactive, and rely on hand-written kernel code. This proposal designs a self-evolving, in-kernel *eBPF network intrusion prevention system* autonomously maintained by Large Language Model coding agents. Inspired by “learning beyond gradients”, we propose the *Defensive Evolution* paradigm: defensive capability is accumulated as eBPF code proven safe by the kernel verifier rather than as neural-network weights. A coding agent explores the attack surface under novelty- and coverage-guided exploration, evolutionarily synthesizes defenses, and hot-loads them through a compile-verify-sandbox trusted loop, evolving the defense library continuously and auditably. Turning the author’s offensive off-path expertise into defense, we build an attack-defense co-evolution evaluation to quantify protection against known and unknown attacks, false positives, and runtime overhead.

Keywords: network protocol security; eBPF; LLM agents; Defensive Evolution; self-evolving systems; intrusion prevention.

目录

一、选题背景与意义	3
1.1 研究背景	3
1.2 研究意义	3
二、国内外研究现状综述	4
2.1 链外协议攻击及其防御: 攻防同源、防御滞后	4
2.2 eBPF 网络安全与运行时防御: 载体成熟、智能化不足	4
2.3 大模型编码智能体用于安全: 能力涌现、面向防御者尚少	5
2.4 启发式/进化式学习与自进化智能体: 方法论根基	5
三、研究内容与拟达到的目标	6
3.1 总体目标	6
3.2 主要研究内容	6
3.3 拟达到的目标	6
四、拟解决的关键问题	6
五、研究方法与技术路线	7
5.1 总体技术路线	7
5.2 防御演化闭环 (核心方法)	7
5.3 关键方法设计	7
5.4 实验设计与系统演示	8
六、可行性分析	8
七、工作特色与创新点	8
八、预期成果	9
九、论文工作计划与进度安排	9
参考文献	13

一、选题背景与意义

1.1 研究背景

TCP/IP 协议栈是互联网的基石,但其设计之初对安全性考虑不足,且经过数十年的演进与打补丁,协议实现内部及协议之间形成了大量复杂的跨层交互。近十年来的研究表明,这些跨层交互中的歧义 (ambiguity) 持续地催生新的**链外 (off-path) 攻击**:攻击者无需位于通信路径上、无需窃听报文,仅凭侧信道即可推断 TCP 连接的关键状态 (如序列号),进而劫持、重置或污染连接¹。

这一攻击面呈现出**持续再生**的特征。自 2016 年 Cao 等人揭示 RFC 5961 引入的全局 challenge-ACK 速率限制可被用作侧信道 (CVE-2016-5696) 以来²,学术界沿“IP-ICMP-TCP 跨层交互”这一主线不断发现新漏洞:混合/降级的 IPID 分配侧信道³、绕过 PMTUD 的 IP 分片攻击⁴、NAT 环境下的 TCP 劫持⁵、Wi-Fi 加密帧的帧长侧信道⁶,乃至作者所在课题组在 2025 年 CCS 上发表的、利用路径 MTU 发现 (PMTUD) 打破 IP 地址共享场景下连接隔离的最新攻击⁷。与此同时,DNS 等无连接协议同样不断出现侧信道投毒攻击⁸,基于 IP 伪造的 TCP 载荷可靠注入也被证明可行⁹。

一个尤为值得警惕的规律是:**许多新攻击面恰恰由防御机制本身引入**——challenge-ACK 限速是为抵御盲注入而设,却成了侧信道²;源端口随机化¹⁰是为抵御 off-path 注入而设,却被侧信道反推⁸;PMTUD 是为优化传输而设,却被用于打破连接隔离⁷。这说明协议安全不是“修补一次即告终结”的静态问题,而是随部署环境 (公网 Wi-Fi、VPN、5G CGNAT 等 IP 共享场景) 与防御补丁不断演化的**动态博弈**。

面对层出不穷的协议攻击,现有防御呈现三个结构性缺陷:

1. **零散**。防御以“一种攻击对应一个补丁”的方式分散在标准 (RFC 5961¹¹、RFC 6056¹⁰、BCP38¹²)、内核实现与中间盒中,缺乏统一的、可持续演进的防御载体;
2. **滞后**。检测侧虽已出现 SCENT¹³、SCAD¹⁴ 等系统化挖掘协议侧信道的工具,但其本质仍是对已成形漏洞类别的**被动追赶**;从漏洞披露到内核缓解上线往往周期漫长;
3. **依赖人工编写内核代码**。具体缓解几乎都需安全专家手工设计、提交并等待上游合入。人工编写内核防御面临**响应慢、易出错 (跨层逻辑复杂、补丁本身可能成为新攻击面)**、知识不可**积累**三重困境。

与此同时,内核可编程技术 **eBPF** 的成熟为运行时防御提供了理想载体。eBPF 允许在不修改内核源码、不重启系统的前提下,将自定义程序安全地注入内核热点路径 (XDP/tc/kprobe/LSM 等),已被工业界广泛用于高性能负载均衡¹⁵、DDoS 缓解¹⁶、可扩展防火墙¹⁷与云原生运行时安全¹⁸⁻²⁰。其内置的**验证器 (verifier)** 能在加载前对程序进行形式化的内存安全与终止性证明²¹⁻²²,构成了“让不可信代码安全运行于内核”的可信基础。

另一条技术线索来自人工智能:**大模型编码智能体 (LLM Coding Agent)** 已展现出自主编写、调试与迭代代码的能力,并在程序搜索与算法发现上取得突破——FunSearch 在程序空间做进化搜索发现了更优算法²³,AlphaEvolve 进一步在系统级代码 (数据中心调度、矩阵乘法) 上做出可验证改进²⁴;在安全领域,编码智能体已能自主发现真实软件漏洞²⁵、完成渗透测试²⁶并利用真实漏洞²⁷。尤为关键的是,已有工作 **Kgent/KEN** 证明大模型能够从自然语言合成 eBPF 程序并结合符号执行保证语义正确²⁸,这为“智能体编写内核防御”提供了直接的可行性证据。

1.2 研究意义

综合上述背景,本课题提出并构建一个由**大模型编码智能体自主维护、跑在内核的自进化 eBPF 网络入侵防御系统**。其意义体现在三个层面:

- **理论意义**:针对“防御能力如何持续学习”这一根本问题,本课题提出**防御演化 (Defensive**

Evolution) 范式——将防御能力沉淀为“经验证的内核代码”而非神经网络权重,以编码智能体的非梯度迭代替代反向传播。这一范式受“超越梯度的学习 (Learning Beyond Gradients)”²⁹ 启发,有望规避深度学习在持续学习中的灾难性遗忘问题,并为“代码即可持续进化的知识载体”提供一个安全关键 (security-critical) 领域的实证。

- **现实/工程意义:** 把作者及课题组在 off-path 协议攻击上积累的**攻击者视角**^{1,7} 反向用于防御,使防御系统能够“主动想象攻击面”并提前生成缓解;以 eBPF 作为可热加载、可验证、贴近协议栈的内核防御载体,有望将协议防御从“被动打补丁”转变为“主动、自动、持续的免疫”,显著缩短从新攻击出现到防御上线的周期。
- **学科交叉意义:** 本课题处在**网络协议安全、操作系统内核 (eBPF)、大模型智能体**三个方向的交叉点上,既是对“AI for Security”的具体落地,也为“AI 自主维护安全关键系统”的可靠性 (verifier + 沙箱 + 自动回滚) 提供方法论探索。

二、国内外研究现状综述

本节从四条线索梳理国内外研究现状,并在每条线索末尾指出其局限,从而引出本课题的切入点。

2.1 链外协议攻击及其防御: 攻防同源、防御滞后

攻击侧,off-path TCP/IP 攻击自 Gilad 等确立“仅能伪造源地址、不能窃听”的威胁模型以来³⁰,已发展为协议安全的活跃方向。清华大学徐恪、李琦、冯学伟等所在的网络体系结构与安全课题组,沿“跨层交互歧义”主线连续产出成果:混合 IPID 分配侧信道³、降级 IPID 劫持、PMTUD 分片攻击⁴、ICMP 重定向攻击的“复活”、WPA 环境下的中间人攻击、NAT 网络 TCP 劫持⁵、Wi-Fi 帧长侧信道⁶,并以 CACM 综述将其统一为“TCP/IP 跨层漏洞”框架¹。作者本人参与的 CCS’25 工作进一步揭示:即便是被普遍信任的 PMTUD 机制,在公网 Wi-Fi/VPN/5G 等 IP 地址共享场景下仍会打破连接隔离,使攻击者平均 220 秒、70% 成功率地推断受害连接序列号,50 个真实网络中 38 个脆弱⁷。UCR 的 Qian 团队则在 DNS 方向揭示了 SAD DNS 等侧信道投毒⁸,CISPA 的 Pan 与 Rossow 证明了伪造握手后仍可可靠注入 TCP 载荷⁹。

防御侧,标准化手段 (RFC 5961 加固¹¹、源端口随机化¹⁰、入口过滤 BCP38¹²) 多为“建议非强制”,部署参差,且如前所述常被转化为新的攻击面。系统化检测方面,Cao 等提出 SCENT 将 TCP 侧信道归结为连接间“非干扰性”的违反并半自动挖掘漏洞¹³,Man 等的 SCAD 将其推广为通用自动检测¹⁴;课题组自身也在无连接协议信息泄露上提出了 StateShield 实时防御³¹。

局限与切入点: 现有防御本质是**逐漏洞、逐协议、人工驱动**的;检测工具能“发现”漏洞却不能“自动修复并部署”;防御知识散落在论文与补丁里无法自动积累复用。这正是本课题要以“自主进化的内核防御系统”填补的缺口。

2.2 eBPF 网络安全与运行时防御: 载体成熟、智能化不足

eBPF 已成为内核可编程与运行时安全的事实标准载体。工业界以 Katran¹⁵、Cloudflare L4Drop¹⁶ 在 XDP 层实现高性能负载均衡与 DDoS 缓解;学术界以 bpf-iptables¹⁷ 复刻并加速防火墙语义; Cilium¹⁸、Falco²⁰、Tetragon¹⁹ 则把 eBPF 用于云原生网络策略与运行时安全可观测/强制执行 (Tetragon 可在系统调用完成前同步阻断)。在安全保障方面,eBPF 验证器是关键:PREVAIL 以抽象解释支持任意循环验证²¹,K2 以程序综合生成“安全且高效”的内核扩展³²,Sun 等通过状态嵌入检测验证器自身缺陷²²。同时,eBPF 也被研究为攻击面——跨容器攻击³³、Evil Packet Filter³⁴ 揭示了其武器化风险,反向论证了“eBPF 防御库本身需要被安全地维护”。

局限与切入点: 上述系统的防御策略几乎全部由人工编写; 规则/程序的演进依赖工程师手工迭代。当攻击快速演化时, 人工维护成为瓶颈。eBPF 提供了优秀的“载体”与“安全闸门”, 但缺少一个自主的“维护者”——这正是引入编码智能体的动机。

2.3 大模型编码智能体用于安全: 能力涌现、面向防御者尚少

大模型在安全领域的应用近两年迅速成熟。进攻侧, PentestGPT 实现自主渗透²⁶, LLM 智能体可自主利用真实一日漏洞²⁷, Google 的 Naptime/Big Sleep 让智能体在真实软件中发现此前未知的内存安全漏洞²⁵; 评测侧出现了 Cybench³⁵、NYU CTF Bench³⁶ 等基准。代码维护侧, 自动程序修复 (APR) 综述显示 LLM 已成为该领域主流方法³⁷。与本课题最直接相关的是 **Kgent/KEN**: 它用大模型从自然语言合成 eBPF 程序, 并以符号执行 + 反馈回路保证语义等价, 正确率达 80%²⁸。

在工程载体层面, **通用编码智能体**已走向成熟并产品化: Anthropic 的 Claude Code³⁸、Any-sphere 的 Cursor³⁹、OpenAI 的 Codex CLI⁴⁰、Cognition 的 Devin⁴¹, 以及开源的 SWE-agent⁴²、OpenHands⁴³ 等, 均能在仓库级任务上自主完成“编辑—运行—调试”的闭环, 并依托标准化的智能体—计算机接口 (agent-computer interface) 调用工具。然而它们面向**通用软件工程**, 并不内置内核 eBPF 防御所需的领域能力——内核 verifier 报错的结构化解析、内核态隔离沙箱、攻防靶场、防御档案与探索信号; 直接套用通用智能体难以胜任本任务。这构成本课题的一项关键设计: 与其直接调用通用智能体, 不如把上述领域工具与反馈固化为面向防御演化的**专用编码智能体 harness** (domain-specific agent scaffold)。

局限与切入点: (1) 现有“LLM 写 eBPF”工作 (如 Kgent²⁸) 是**一次性、按需合成**, 而非**长期自主维护一整套防御库 + 运行时反馈 + 持续进化**; (2) 安全应用多集中在**进攻侧**, 面向**防御者**、面向**协议攻击**的自主防御研究尚少; (3) 研究普遍指出 LLM 生成代码的可靠性存疑 (不安全代码比例可达 9.8%–42.1%⁴⁴), 因此“让 AI 写内核代码”必须配套强制的验证与沙箱——这恰是 eBPF verifier 的独特价值所在; (4) 通用编码智能体^{38,42}缺少内核/eBPF 的领域工具与反馈闭环, 需要为本任务设计**专用 harness**。

2.4 启发式/进化式学习与自进化智能体: 方法论根基

本课题“非梯度、持续进化”的方法论有坚实的文献支撑。其直接灵感是“超越梯度的学习”²⁹: 主张编码智能体可通过**持续编辑一个软件系统** (而非更新权重) 实现“学习”, 具备可解释、样本高效、可回归测试、抗灾难性遗忘等优势。在程序搜索层面, FunSearch²³、AlphaEvolve²⁴、EoH⁴⁵ 证明“冻结的大模型作进化算子 + 自动评估器作选择压力”能在程序/系统级代码上做出可验证改进。在自我改进层面, Reflexion⁴⁶ 以语言化反思替代梯度积累经验, Self-Debug⁴⁷ 让模型据执行/编译错误自我修正, Darwin-Gödel Machine⁴⁸ 与 Voyager⁴⁹ 展示了带档案库、可持续自我进化的智能体。在探索机制层面, 基于预测误差与新颖性的探索方法 ICM⁵⁰、随机网络蒸馏 RND⁵¹ 提供了“新颖性/预测误差”驱动的探索信号, 新颖性搜索⁵² 与 MAP-Elites⁵³ 提供了“维护多样化优质解集”的算法范式, 其思想可追溯至 Schmidhuber 关于内在激励与新颖性的早期理论⁵⁴。

在自主网络防御 (ACD) 方向, 自主计算⁵⁵ 与自保护软件系统⁵⁶ 奠定了 MAPE-K 自适应闭环的理论基础, CybORG/CAGE⁵⁷ 提供了 RL 防御基准, 近期亦有将大模型用作自主防御者的探索⁵⁸。

局限与切入点: 这些方法论尚未被系统性地应用于“内核网络防御”这一安全关键、强可验证、反馈廉价的场景。本课题正是要把“新颖性与覆盖度引导的主动探索 + LLM 进化式程序合成 + verifier/沙箱选择压力 + 新颖性档案”整合为面向 eBPF 防御的**防御演化闭环**。

三、研究内容与拟达到的目标

3.1 总体目标

设计并实现一个**自进化 eBPF 网络入侵防御系统**: 由大模型编码智能体在”防御演化”范式下, 自主地发现网络协议攻击面、合成并迭代 eBPF 防御程序、经验证与沙箱评估后热加载到内核数据面, 形成一个**持续生长、可审计、形式化可验证**的防御代码库, 在保持低误报与线速性能的同时, 逐步提升对已知与未知协议攻击的防御能力。

3.2 主要研究内容

研究内容一: 面向 eBPF 防御的完整专用 harness。将”由编码智能体生成 eBPF 防御并保证其可用、安全”的全过程, 封装为一套完整的领域专用 harness: 涵盖防御程序的生成、编译、内核 verifier 校验、隔离沙箱测试、加载与运行时反馈接口; 并解决两个关键工程问题——生成质量(智能体生成的 eBPF 能编译通过、通过 verifier, 且确实拦截攻击而不误伤正常流量)与运行安全(以编译/verifier 报错驱动自我修正, 异常时自动回滚, 保证不致网络中断或误伤正常流量)。这区别于直接调用 Claude Code、Cursor 等通用编码智能体^{38-39,42}。

研究内容二: 面向真实协议攻击的防御生成与评测。复用 PMTUD/NAT/IPID 等真实协议攻击^{3,5,7}搭建攻防靶场, 让系统针对这些攻击自动生成并迭代防御; 量化检测率、误报率、XDP 吞吐与时延开销, 以及达到目标防御所需的人工介入量, 并与人工编写规则、Kgent²⁸、Tetragon¹⁹/Falco²⁰等对比; 对未让系统接触过的新攻击单独测试, 考察其泛化能力。

3.3 拟达到的目标

- 完成系统原型与智能体工具链闭环(在 PacketScope/Guarder 上落地);
- 在典型协议攻击集合(含课题组自有攻击)上, 系统在**无人工编写规则**前提下自主生成有效 eBPF 防御, 关键攻击的防御成功率与人工专家规则相当或更优, 误报率控制在可用范围;
- 验证”自进化”特性: 面对**预留的未知攻击**, 系统能在无人工补丁介入下经若干轮迭代自主提升防御覆盖;
- XDP 数据面在开启防御时的吞吐损失与时延开销维持在工程可接受水平;
- 形成一套可运行、可复现的开源系统与攻防评测靶场。

四、拟解决的关键问题

1. **可靠性——如何让智能体自动生成的内核代码安全可控?** 内核态 eBPF 一旦出错可致网络中断或误伤全部流量, 而 LLM 生成代码可靠性存疑⁴⁴。关键在于把**现成的内核安全设施**(内核 verifier、静态沙箱、隔离环境对抗验证、自动回滚) **接入智能体闭环**: 以编译/verifier 报错驱动自修正, 沙箱重放把关, 异常即回滚——本文不重造这些基础设施, 而是组织它们构成多层可靠闭环。
2. **生成质量——智能体生成的 eBPF 怎样才”能用”?** 它必须能编译通过、通过内核 verifier、且确实拦截目标攻击而不误伤正常流量。关键在于把编译/verifier 报错与沙箱重放结果作为结构化反馈, 驱动智能体迭代修正, 直到产出可用且达标的防御。
3. **评测——如何证明系统”真的有用”?** 不仅要防住已见过的攻击, 对未让系统接触过的新攻击也要能防。关键在于构建真实攻击靶场, 给出检出率、误报率、性能开销与人工介入量等可复现、可对比的度量, 并与人工规则及现有工具对比。

五、研究方法与技术路线

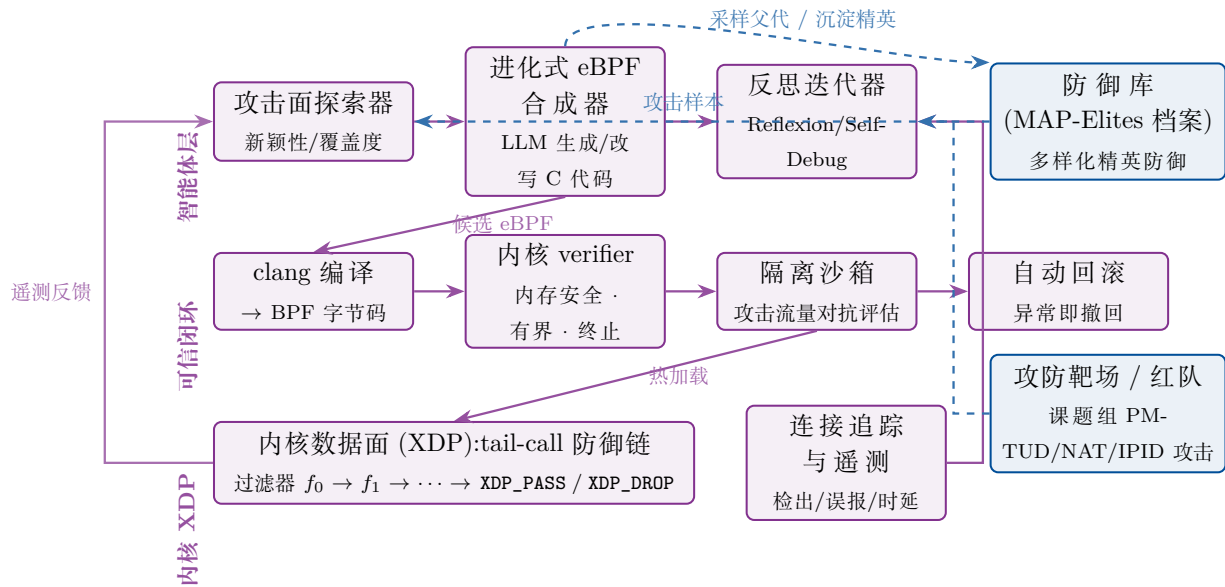


图 1: 自进化 eBPF 网络入侵防御系统总体架构: 编码智能体在“探索—合成—可信验证—部署反馈—档案”闭环中维护内核防御库。

5.1 总体技术路线

本课题采用“理论建模 → 系统实现 → 攻防评测 → 迭代改进”的研究方法, 以课题组开源项目 PacketScope 的 Guarder 模块为工程基线 (其已具备 XDP 数据面、cilium/ebpf 一键编译、规则热加载、MCP 接入与多维遥测, 并已有“Agent 可编程 eBPF + 四层安全模型”的原型), 逐步构建完整的防御演化闭环。

5.2 防御演化闭环 (核心方法)

系统以一个开放式进化循环运行, 各环节均有方法论依据:

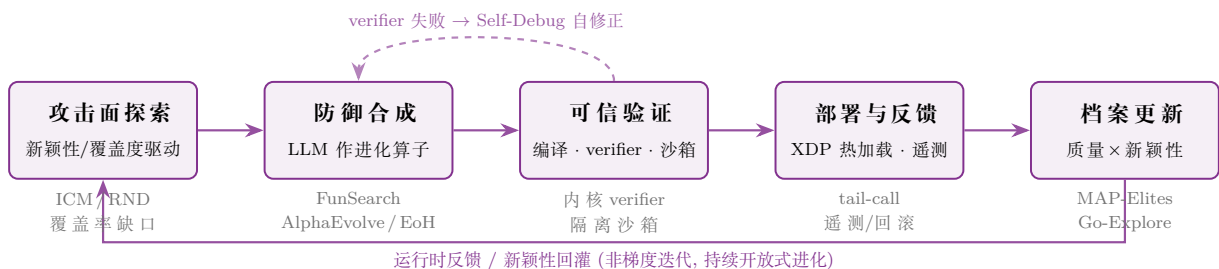


图 2: 防御演化 (Defensive Evolution) 闭环: 以非梯度方式持续进化内核 eBPF 防御。

5.3 关键方法设计

- 可信闭环 (对应关键问题 1): 四层防御——静态沙箱 (禁危险 helper、禁未授权 map 声明、限制循环/规模)→ 模板封装 (智能体代码注入安全 wrapper)→ 编译器 ‘-Wall -Werror -target

bpf^{*} → 内核 verifier; 部署后以”丢包异常飙升/健康检查失败”触发自动回滚, 关键管理端口 (如 SSH) 加白名单保护; 全部变更落 git 审计。

- **鲁棒探索 (对应关键问题 2):** 优先采用对随机性鲁棒的 RND⁵¹/伪计数与信息增益信号, 而非朴素预测误差; 以”防御覆盖率缺口”为显式探索目标。
- **抗腐化进化 (对应关键问题 3):** 每条已防住的攻击固化为回归用例, 智能体每次改动须通过全量攻击回放方可合入; 周期性触发”历史压缩”重构, 将多条具体特征归并为通用检测原语, 以耦合复杂度 (coupling complexity) 作为系统健康度指标。
- **评测协议 (对应关键问题 4):** 构建包含已知攻击与预留的未知攻击的攻防靶场; 以攻防协同进化 (借鉴 POET 思想) 生成递增难度的攻击场景; 借助 Cybench/NYU-CTF³⁵⁻³⁶ 思路量化智能体的攻击面理解能力。

5.4 实验设计与系统演示

在受控攻防靶场中部署 victim/attacker/puppet/网关与本系统, 复现课题组 PMTUD⁷、NAT 劫持⁵、IPID³ 等攻击作为红队, 度量: 对已知攻击的检出率/误报率; 对未知攻击的自主防御提升 (迭代曲线); XDP 吞吐/时延开销; 自动化程度 (人工介入量)。与 Kgent²⁸ (LLM 写 eBPF)、Tetragon¹⁹/Falco²⁰ (人工策略运行时防御) 正面对比。按计算机系答辩要求, 提供可测试的典型系统演示 (智能体自主从”被某攻击绕过”到”生成有效 eBPF 防御”的端到端闭环)。

六、可行性分析

1. **理论可行:** 防御演化的方法论根基已被顶刊/顶会验证——FunSearch(Nature)²³、AlphaEvolve²⁴ 证明 LLM 进化式程序搜索可在系统级代码上做出可验证改进; Darwin-Gödel Machine⁴⁸ 证明自进化智能体 + 档案库 + 沙箱可行; Kgent²⁸ 证明 LLM 能合成正确 eBPF。
2. **工程可行:** 课题组已有开源系统 **PacketScope/Guarder** 作为基线, 具备 XDP 数据面、cilium/ebpf(bpf2go CO-RE) 一键编译、规则/程序热加载、MCP 接入、多维遥测, 并已在作者的前期工作中实现”Agent 可编程 eBPF 过滤器 + tail-call 热加载 + 四层安全模型”原型, 且通过了 312 个测试用例, **可行性已被前期原型初步证明**。
3. **数据/攻击样本可行:** 作者及课题组在 off-path 协议攻击上的系列工作^{1,3-7} 提供了高质量、可复现的真实攻击样本与攻防靶场, 是本课题”红队/种子攻击”的独特优势。
4. **安全可控:** 内核 eBPF verifier 提供了”不可伪造”的形式化安全闸门, 配合隔离沙箱与自动回滚, 使”让 AI 写内核代码”的风险可控。
5. **条件可行:** 导师徐恪教授课题组在网络协议安全方向具备深厚积累与算力/实验条件; 作者已完成相关攻击研究与系统原型, 具备 eBPF、内核与大模型智能体开发能力。

主要风险与对策: 大模型生成内核代码的稳定性——以 verifier+ 沙箱 + 回归测试兜底, 必要时引入”浅层模型做感知 + 启发式做规则”的混合; 自动闭环在真实噪声下未必如靶场般干净——以鲁棒新颖性信号 + 人工门控分级放行缓解; 未知攻击的”未知”难以穷尽——以预留测试集 + 协同进化近似, 并如实报告覆盖边界。

七、工作特色与创新点

1. **可运行的系统 (而非设想):** 在 PacketScope/Guarder 上落地一套由编码智能体自动维护的内核 eBPF 防御系统, 可演示、可复现。

2. **智能体写内核代码的可靠流水线:** 把”生成 → clang 编译 → 内核 verifier → 隔离沙箱 → tail-call 热加载 → 自动回滚”组织成一条可靠流水线, 系统化解决”让 AI 写内核代码”的可用性与安全性 (其中 verifier、沙箱、回滚等均为现成基础设施, 本文的工作是把它们与编码智能体闭环组织起来, 而非重新发明)。
3. **面向真实协议攻击的防御评测靶场:** 以 PMTUD/NAT/IPID 等真实攻击构建攻防靶场, 给出检出率、误报率、性能开销与人工介入量等量化结果, 并与人工规则、Kgent²⁸、Tetragon¹⁹/Falco²⁰ 对比; 靶场本身可复用于同类内核安全研究。

八、预期成果

1. **学位论文一篇,** 系统阐述防御演化范式、自进化 eBPF 防御系统的设计、实现与评测;
2. **开源系统:** 在 PacketScope 基础上发布自进化 eBPF 防御系统原型、面向防御演化的**专用编码智能体 harness**(领域工具 + 闭环编排) 与攻防评测靶场;
3. **可能的标准化/披露贡献:** 若发现并自动缓解新协议攻击, 向社区 (IETF/Linux 等) 负责任披露。

九、论文工作计划与进度安排

阶段	时间	主要工作	阶段成果
开题	2026.06	选题报告与开题答辩; 确定范式与系统框架	选题报告、开题答辩
一	2026.07–2026.09	防御演化范式形式化; 在 PacketScope/Guarder 上搭建系统总体框架与闭环骨架	系统框架、原型 v0
二	2026.10–2026.12	新颖性与覆盖度引导的主动探索 + LLM 进化式 eBPF 合成 + verifier/沙箱可信加载闭环	闭环原型 v1、初步实验
三	2027.01–2027.03	多样化防御库维护; 攻防协同进化评测; 与 Kgent/Tetragon/Falco 对比; 中期检查	完整系统、评测结果、中期报告
四	2027.04–2027.05	论文撰写、投稿; 系统完善; 预答辩与盲审送审	学位论文初稿、投稿论文
五	2027.06	论文修改、答辩与学位申请	学位论文、答辩

参考文献

[1] FENG X, LI Q, SUN K, et al. Exploiting Cross-Layer Vulnerabilities: Off-Path Attacks on the TCP/IP Protocol Suite[J]. Communications of the ACM, 2025, 68(3): 48-59. DOI: [10.1145/3689819](#).

[2] CAO Y, WANG Z, QIAN Z, et al. Off-Path TCP Exploits: Global Rate Limit Considered Dangerous[C]//25th USENIX Security Symposium. 2016: 209-225.

- [3] FENG X, FU C, LI Q, et al. Off-Path TCP Exploits of the Mixed IPID Assignment[C]//Proc. 2020 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2020: 1323-1335. DOI: [10.1145/3372297.3417884](https://doi.org/10.1145/3372297.3417884).
- [4] FENG X, LI Q, SUN K, et al. PMTUD is not Panacea: Revisiting IP Fragmentation Attacks against TCP[C]//29th Network and Distributed System Security Symposium (NDSS). 2022. DOI: [10.14722/ndss.2022.24381](https://doi.org/10.14722/ndss.2022.24381).
- [5] YANG Y, FENG X, LI Q, et al. Exploiting Sequence Number Leakage: TCP Hijacking in NAT-Enabled Wi-Fi Networks[C]//31st Network and Distributed System Security Symposium (NDSS). 2024.
- [6] WANG Z, FENG X, LI Q, et al. Off-Path TCP Hijacking in Wi-Fi Networks: A Packet-Size Side Channel Attack[C]//32nd Network and Distributed System Security Symposium (NDSS). 2025.
- [7] FENG X, LI Z, LI Q, et al. Off-Path TCP Exploits: PMTUD Breaks TCP Connection Isolation in IP Address Sharing Scenarios[C]//Proc. 2025 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2025. DOI: [10.1145/3719027.3744888](https://doi.org/10.1145/3719027.3744888).
- [8] MAN K, QIAN Z, WANG Z, et al. DNS Cache Poisoning Attack Reloaded: Revolutions with Side Channels[C]//Proc. 2020 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2020: 1337-1350. DOI: [10.1145/3372297.3417280](https://doi.org/10.1145/3372297.3417280).
- [9] PAN Y, ROSSOW C. TCP Spoofing: Reliable Payload Transmission Past the Spoofed TCP Handshake[C]//2024 IEEE Symposium on Security and Privacy (SP). 2024.
- [10] LARSEN M, GONT F. Recommendations for Transport-Protocol Port Randomization[Z]. RFC 6056 (BCP 156), IETF. 2011.
- [11] RAMAIAH A, STEWART R, DALAL M. Improving TCP's Robustness to Blind In-Window Attacks[Z]. RFC 5961, IETF. 2010.
- [12] FERGUSON P, SENIE D. Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing[Z]. RFC 2827 (BCP 38), IETF. 2000.
- [13] CAO Y, WANG Z, QIAN Z, et al. Principled Unearthing of TCP Side Channel Vulnerabilities[C]//Proc. 2019 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2019: 211-224. DOI: [10.1145/3319535.3354250](https://doi.org/10.1145/3319535.3354250).
- [14] MAN K, WANG Z, HAO Y, et al. SCAD: Towards a Universal and Automated Network Side-Channel Vulnerability Detection[C]//33rd USENIX Security Symposium. 2024.
- [15] SHIROKOV N, DASINENI R. Open-sourcing Katran, a Scalable Network Load Balancer[Z]. Meta Engineering Blog. 2018.
- [16] FABRE A, MAJKOWSKI M. L4Drop: XDP DDoS Mitigations[Z]. The Cloudflare Blog. 2018.
- [17] MIANO S, BERTRONE M, RISSO F, et al. Securing Linux with a Faster and Scalable Iptables[J]. ACM SIGCOMM Computer Communication Review, 2019, 49(3). DOI: [10.1145/3371927.3371929](https://doi.org/10.1145/3371927.3371929).
- [18] GRAF T. EBPF – The Future of Networking and Security[Z]. Cilium / Isovalent Blog. 2020.

- [19] Isovalent. Tetragon: eBPF-based Security Observability and Runtime Enforcement[Z]. Isovalent / CNCF. 2022.
- [20] The Falco Authors. Falco: Cloud Native Runtime Security[Z]. CNCF Project. 2018.
- [21] GERSHUNI E, AMIT N, GURFINKEL A, et al. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions[C]//Proc. 40th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI). 2019: 1069-1084. DOI: [10.1145/3314221.3314590](https://doi.org/10.1145/3314221.3314590).
- [22] SUN H, SU Z. Validating the eBPF Verifier via State Embedding[C]//18th USENIX Symp. on Operating Systems Design and Implementation (OSDI). 2024: 615-628.
- [23] ROMERA-PAREDES B, BAREKATAIN M, NOVIKOV A, et al. Mathematical Discoveries from Program Search with Large Language Models[J]. Nature, 2024, 625: 468-475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- [24] NOVIKOV A, VŮ N, EISENBERGER M, et al. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery[J]. ArXiv preprint arXiv:2506.13131, 2025.
- [25] GLAZUNOV S, BRAND M. Project Naptime: Evaluating Offensive Security Capabilities of Large Language Models[Z]. Google Project Zero Blog. 2024.
- [26] DENG G, LIU Y, MAYORAL-VILCHES V, et al. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing[C]//33rd USENIX Security Symposium. 2024: 847-864.
- [27] FANG R, BINDU R, GUPTA A, et al. LLM Agents can Autonomously Exploit One-day Vulnerabilities[J]. ArXiv preprint arXiv:2404.08144, 2024.
- [28] YANG Y, ZHENG Y, CHEN M, et al. Kgent: Kernel Extensions Large Language Model Agent[C]//Proc. ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions. 2024: 30-36. DOI: [10.1145/3672197.3673434](https://doi.org/10.1145/3672197.3673434).
- [29] WENG J. Learning Beyond Gradients[Z]. <https://github.com/Trinkle23897/learning-beyond-gradients>. 2026.
- [30] GILAD Y, HERZBERG A. Off-Path Attacking the Web[C]//6th USENIX Workshop on Offensive Technologies (WOOT). 2012.
- [31] LI H, LI Q, SU Y, et al. StateShield: Real-Time Defenses Against Information Leakage over Connectionless Protocols[J]. IEEE/ACM Transactions on Networking, 2025.
- [32] XU Q, WONG M D, WAGLE T, et al. Synthesizing Safe and Efficient Kernel Extensions for Packet Processing[C]//Proc. 2021 ACM SIGCOMM Conference. 2021. DOI: [10.1145/3452296.3472929](https://doi.org/10.1145/3452296.3472929).
- [33] HE Y, GUO R, XING Y, et al. Cross Container Attacks: The Bewildered eBPF on Clouds[C]//32nd USENIX Security Symposium. 2023: 5971-5988.
- [34] JIN D, ATLIDAKIS V, KEMERLIS V P. EPF: Evil Packet Filter[C]//2023 USENIX Annual Technical Conference (ATC). 2023: 735-751.
- [35] ZHANG A K, PERRY N, DULEPET R, et al. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models[C]//International Conference on Learning Representations (ICLR). 2025.

- [36] SHAO M, JANCHESKA S, UDESHI M, et al. NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security[C]//Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track. 2024: 57472-57498.
- [37] ZHANG Q, FANG C, XIE Y, et al. A Systematic Literature Review on Large Language Models for Automated Program Repair[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2024.
- [38] Anthropic. Claude Code: Agentic Coding in the Terminal[Z]. <https://www.anthropic.com/claude-code>. 2025.
- [39] AnySphere. Cursor: The AI Code Editor[Z]. <https://www.cursor.com>. 2024.
- [40] OpenAI. Codex CLI: A Lightweight Coding Agent for the Terminal[Z]. <https://github.com/openai/codex>. 2025.
- [41] Cognition AI. Devin: An AI Software Engineer[Z]. <https://www.cognition.ai/blog/introducing-devin>. 2024.
- [42] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[C]//Advances in Neural Information Processing Systems (NeurIPS). 2024.
- [43] WANG X, LI B, SONG Y, et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents[C]//International Conference on Learning Representations (ICLR). 2025.
- [44] Anonymous. An Empirical Study on LLM-Driven Secure Code Generation[J]. ArXiv preprint arXiv:2506.23034, 2025.
- [45] LIU F, XIALIANG T, YUAN M, et al. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model[C]//Int. Conf. on Machine Learning (ICML). 2024.
- [46] SHINN N, CASSANO F, LABASH B, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[C]//Advances in Neural Information Processing Systems (NeurIPS). 2023.
- [47] CHEN X, LIN M, SCHÄRLI N, et al. Teaching Large Language Models to Self-Debug[C]//International Conference on Learning Representations (ICLR). 2024.
- [48] ZHANG J Z, HU S, LU C, et al. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents[J]. ArXiv preprint arXiv:2505.22954, 2025.
- [49] WANG G, XIE Y, JIANG Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[C]//ArXiv preprint arXiv:2305.16291. 2023.
- [50] PATHAK D, AGRAWAL P, EFROS A A, et al. Curiosity-driven Exploration by Self-supervised Prediction[C]//Int. Conf. on Machine Learning (ICML). 2017.
- [51] BURDA Y, EDWARDS H, STORKEY A, et al. Exploration by Random Network Distillation[C]//International Conference on Learning Representations (ICLR). 2019.

- [52] LEHMAN J, STANLEY K O. Abandoning Objectives: Evolution Through the Search for Novelty Alone[J]. *Evolutionary Computation*, 2011, 19(2):189-223. DOI: [10.1162/EVCO_a_00025](https://doi.org/10.1162/EVCO_a_00025).
- [53] MOURET J B, CLUNE J. Illuminating Search Spaces by Mapping Elites[J]. *ArXiv preprint arXiv:1504.04909*, 2015.
- [54] SCHMIDHUBER J. Formal Theory of Creativity, Fun, and Intrinsic Motivation (1990–2010)[J]. *IEEE Transactions on Autonomous Mental Development*, 2010, 2(3):230-247. DOI: [10.1109/TAMD.2010.2056368](https://doi.org/10.1109/TAMD.2010.2056368).
- [55] KEPHART J O, CHESS D M. The Vision of Autonomic Computing[J]. *Computer*, 2003, 36(1):41-50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
- [56] YUAN E, ESFAHANI N, MALEK S. A Systematic Survey of Self-Protecting Software Systems[J]. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 2014, 8(4):1-41. DOI: [10.1145/2555611](https://doi.org/10.1145/2555611).
- [57] STANDEN M, LUCAS M, BOWMAN D, et al. CybORG: A Gym for the Development of Autonomous Cyber Agents[C]//IJCAI-21 1st Int. Workshop on Adaptive Cyber Defense. 2021.
- [58] CASTRO S, et al. Large Language Models are Autonomous Cyber Defenders[J]. *ArXiv preprint arXiv:2505.04843*, 2025.